EHR Media Intelligence Platform - Write-up

Repo: github.com/usamamunawar118/EHR | Python 3.11+, FastAPI, Pydantic v2, fhir.resources (R4B), sentence-transformers + Chroma, Claude (Haiku 5.5) | 354 Python + 28 JS tests, 95% coverage | all data synthetic

Tradeoffs I made, and why

- **FHIR R4B, not R4.** fhir.resources 8.x, the only Pydantic v2 line, ships R5, R4B and STU3 but no pure R4. R4B is wire-compatible for the five resources I map; the pure-R4 line (6.x) needs Pydantic v1, which the required stack rules out.
- **Three-layer FHIR validation.** The library checks structure only (it accepted gender="banana"), so I added reference-integrity and required-binding / profile checks. Cost: not full HL7 conformance (no FHIRPath invariants or terminology server).
- **Quarantine, never merge.** One MRN shared by different people is quarantined for human review; the same name and DOB under two MRNs is flagged, not merged. A wrong merge is a patient-safety event; the cost is missing patients until reviewed.
- **Do not trust the prompt alone.** Anomalies (abnormal labs, data-quality tags) are detected in code, given to the model, and re-added by a guardrail if dropped. The 200-word limit is enforced in code (target 150, one retry, then trim); the disclaimer and provenance come from code, not the model. More code than a prompt, but every guarantee is testable.
- **Privacy by design.** The model sees age and sex, never name, MRN or exact date of birth; record text is wrapped as untrusted data to resist prompt injection.
- **Search.** Chroma over FAISS for native metadata filters (its range filters are numeric-only, so dates are stored as YYYYMMDD integers). AI-summary vectors are tagged resource_type=Patient and dated by the latest record, so one filter model covers records and summaries; an imaging DocumentReference and its DiagnosticReport collapse into one hit.
- **Front end.** Vanilla JS with committed Tailwind CSS (no Node needed to run) and a native dialog for focus trap and Esc. Cost: hand-written rendering code.

What I would improve with more time

- Validate with the official HL7 validator and add Condition resources coded in ICD-10 / SNOMED. Real OCR and layout parsing instead of simulated scans. A review UI for the quarantine / master-patient-index workflow.
- Authentication (SSO), an access audit log, encryption at rest and a BAA / zero-retention agreement before any real PHI - the model currently receives clinical text.
- Hybrid BM25 + vector retrieval with a cross-encoder reranker; sentence-level citations from each summary claim to its source passage; the Batch API for bulk summaries; Docker packaging; a clinician-reviewed gold set to measure summaries properly.

FHIR and clinical concepts I had to research

- R4 vs R4B and which library versions ship which; Bundle types and in-bundle urn:uuid references; DocumentReference (a pointer to a document) vs DiagnosticReport (a result set referencing Observations); why every document and result needs an Encounter; LOINC codes (verified against the NLM registry) and UCUM units; identifier type MR; Encounter class (v3-ActCode) and observation interpretation H / L / N; MRN canonicalisation and OCR confusions (O/0, l/1); duplicate-patient vs identifier-conflict handling; HIPAA minimum-necessary (age, not date of birth).

How I validated AI summary quality

- **1. Schema:** the model returns constrained JSON. **2. Deterministic scoring** of every summary (ehr eval-summaries, 60 patients): at most 200 words, disclaimer present, *grounded* (every date, ICD-10 code and multi-digit number traceable to the source), and recall of system-detected anomalies and coded diagnoses. **3. Tests that the guardrails fire:** over-length gets a retry then a trim; a dropped anomaly is restored; an invented value lowers confidence; a rounded value (10 for 10.2) is caught - this test found a bug in my first grounding check. **4. A manual rubric** (docs/summary_review_rubric.md): accuracy, no invention, completeness, calibration, safety.
- **Result and honest limits.** The offline extractive baseline scores 100% on word limit, disclaimer, grounding, anomaly recall and diagnosis recall across 60 synthetic patients. I had no API key while building, so **Claude's own output was not scored**: the adapter is unit-tested against a stubbed SDK, and pytest -m live runs the same harness on real output. The rubric review of real output is still to do. These checks catch invented facts, not clinical misjudgement.

Measured

- Search over HTTP with the real MiniLM model, 60 patients / 404 index entries, 32 queries: median 84 ms, p95 109 ms, max 137 ms (requirement: under 2 s). UI: axe-core reports 0 violations (WCAG 2.x A/AA) in the initial, results and drawer states; verified at phone width and keyboard-only.